
qlib Integration

Return & risk prediction, portfolio optimization, and score enhancement

AI Analyst embeds Microsoft `qlib` as a lean, in-process library to power its quantitative desk: a machine-learned return forecast, a factor-structured forward covariance, portfolio optimization, and a walk-forward backtest, with the learned signal folded into the idea scanner and the investment committee. This volume documents the *engineering* of that integration — the lean-embed principle, the module map, the Postgres-to-qlib panel adapter, the dual optimizer dispatcher, the committee and scanner wiring, the REST surface, and operations. The *mathematics* of each model is specified in the *Quantitative & ML Models* volume.

Contents

1	What the integration delivers	2
2	How qlib maps onto the desk's needs	2
3	Module map	3
4	The panel adapter	3
5	Dual optimizer dispatch	3
6	Score enhancement: α + regime-IC weights	4
7	Where the macro-regime IC feeds forecasting	4
8	Committee integration	4
9	REST API and frontend	5
10	Operations	5

1 What the integration delivers

The quantitative desk adds a machine-learned layer on top of the deterministic valuation engine and the fundamentals warehouse:

- **Return prediction** — a cross-sectional LightGBM alpha model over the monthly fundamentals panel, producing a per-name expected forward return (the `mu` consumed by the optimizer and the scanner).
- **Risk prediction** — a factor-structured forward covariance (`StructuredCovEstimator`), $\Sigma = F \Sigma_B F^T + \text{diag}(\mathbf{d})$, with its decomposition exposed to the enhanced-indexing optimizer.
- **Portfolio optimization** — both a native optimizer *and* qlib's optimizers, selectable at runtime.
- **Score enhancement** — the learned alpha plus regime-conditioned factor-IC weights blended into the value + sentiment interest score.
- **Committee integration** — a `quant_signals` evidence node and a Quantitative Factor Analyst specialist.
- **REST + UI** — a `/api/quant/*` router and a *Quant* tab.

Everything degrades gracefully: with no trained model, or with qlib or an optional dependency absent, each surface returns a defined fallback and the application runs unchanged.

The lean-embed principle

qlib's model, risk estimator, optimizers, and evaluation run as *plain libraries* on a MultiIndex DataFrame the app builds from Postgres. The integration never uses qlib's expression engine, its `.bin` data provider, the Alpha158/360 handlers, or the recorder/`qrun` workflow — so there is no `qlib.init`, no data-provider bootstrap, and no mlflow tracking backend. Only `lightgbm`, `scikit-learn`, and `cvxpy` become hard new runtime dependencies.

2 How qlib maps onto the desk's needs

Need	qlib capability used	Module
Return prediction	LGBModel + DatasetH + DataHandlerLP.from_df	api/quant/qlib_alpha.py
Risk prediction	StructuredCovEstimator	api/quant/qlib_risk.py
Portfolio optimization	PortfolioOptimizer, EnhancedIndexingOptimizer	api/quant/qlib_optimize.py
Evaluation / backtest	calc_ic, risk_analysis	api/quant/qlib_backtest.py

3 Module map

File	Responsibility
<code>qlib_data.py</code>	Build the monthly cross-sectional feature/label panel from Postgres (reuses the regime-IC loaders and point-in-time alignment).
<code>qlib_alpha.py</code>	AlphaLGB model (recorder-free), <code>train/predict/evaluate</code> , persistence, lazy cache, CLI.
<code>qlib_risk.py</code>	<code>structured_cov()</code> → annualized Σ + the $(F, \Sigma_B, \mathbf{d})$ decomposition; sync price/returns loader.
<code>qlib_optimize.py</code>	<code>solve(optimizer, ...)</code> dispatcher over the native + five qlib backends → a uniform <code>PortfolioSolution</code> .
<code>qlib_backtest.py</code>	Walk-forward out-of-sample signal backtest scored with <code>qlib_risk_analysis</code> .
<code>alpha_signal.py</code>	Central bridge: cached latest cross-section + per-ticker expected returns (used by scanner, committee, router).
<code>ic_weights.py</code>	Regime-conditioned factor-IC → per-family scoring weights.
<code>optimize.py</code> / <code>risk.py</code>	Native SLSQP/MIP optimizer; sample / Ledoit-Wolf / FF structured covariance.
<code>routers/quant.py</code>	<code>/api/quant/{backends,alpha,risk,optimize,backtest}</code> .

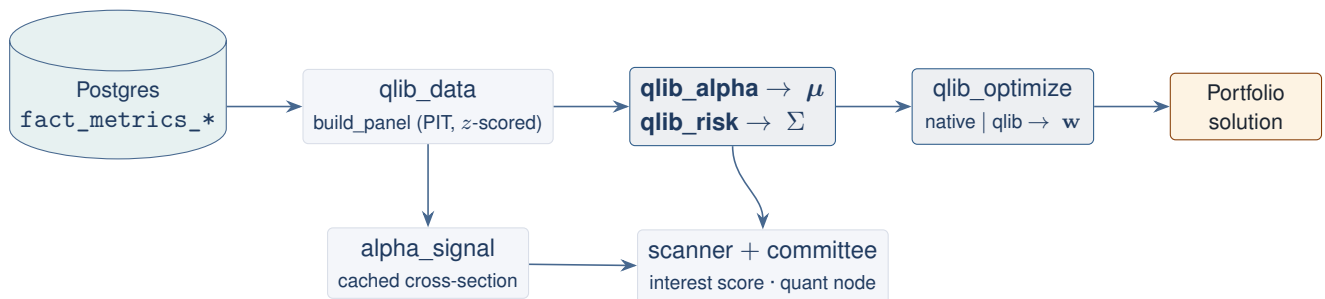


Figure 1. The lean qlib embed. One point-in-time panel feeds the alpha and risk models; the optimizer dispatcher consumes their μ and Σ ; a cached bridge exposes the alpha to the scanner and committee.

4 The panel adapter

`qlib_data.build_panel` produces a qlib-style `MultiIndex` frame — rows (`datetime, instrument`), columns (`feature|label`) — consumed directly by `DataHandlerLP.from_df`. Features are importance-ranked fundamentals from four families, cross-sectionally z -scored per month; the label is the forward 1m/3m return; fundamentals are aligned point-in-time (`period_end + 90d` → month-end). This is the *same* loader and alignment the regime-IC research uses, so the production forecast and the research signal share one lookahead-safe data path. The alpha model trains through the real qlib `DatasetH` pipeline; the `AlphaLGB` subclass attaches `LightGBM`'s callbacks directly in `fit`, so training runs with plain `LightGBM` and needs no `qlib.init` or tracking backend. Prediction runs the stored booster on any reindexed feature matrix.

5 Dual optimizer dispatch

`qlib_optimize.solve(optimizer, tickers, mu, sigma, ...)` dispatches to either backend family and returns a uniform `PortfolioSolution` (weights, annualized expected return / vol / Sharpe, factor exposures, diagnostics):

optimizer	Engine	Notes
native	optimize.py (SLSQP / cvxpy-MIP)	Full constraints: sector caps, vol cap, cardinality, turnover.
qlib_mvo / _gmw / _rp / _inv	qlib PortfolioOptimizer	Mean-variance / min-variance / risk-parity / inverse-vol; long-only, fully invested.
qlib_enhanced_indexing	qlib EnhancedIndexingOptimizer	Benchmark-relative tracking-error; consumes the StructuredCovEstimator decomposition.

Both families are fed the same annualized μ (alpha or historical mean) and Σ (qlib-structured, Ledoit–Wolf, or sample). Neither is a default-only path — both ship enabled and are chosen per request or from the UI selector.

6 Score enhancement: alpha + regime-IC weights

The value + sentiment interest score (`screeener_agent`) blends two learned inputs on top of the fundamental composite:

1. **Normalized alpha** v_α — the qlib model’s expected return, cross-sectionally normalized, as the dominant term (default weight 0.40) when the model covers the shortlist.
2. **Regime-IC family weights** — the fundamental score budget (value / growth) is re-split by how predictive each family has been *in the current macro regime* (§7), rather than by fixed constants.

When no trained model or IC data is available the score reduces exactly to the fixed-weight composite (guaranteed by test). Each row reports `alpha`, `alpha_percentile`, and a `score_components` breakdown; the response carries a `scoring` block naming the weights and model actually used.

7 Where the macro-regime IC feeds forecasting

The macro-cycle model (HMM / VAE) labels each month with a regime; the regime-IC engine then computes, *conditioned on that regime*, the Spearman rank IC of every fundamental metric against forward returns, into `fact_equity_factor_ic_regime`. The one live consumption path in the desk is scoring: `ic_weights.family_weights` reads the latest regime’s rows, aggregates per-metric |IC| to family-level emphasis, and the scanner re-splits its fundamental budget accordingly — so the interest score leans harder on whichever factor family the current regime says is most predictive.

Scope

The LightGBM alpha model itself is regime-*agnostic*: it learns from factor *values* (including momentum/volatility from the market-factor family), and the regime enters forecasting through the score weights, not as a model feature. The committee receives macro regime context from the macro node and the `quant_signals` block, but not the factor-IC table directly.

8 Committee integration

- **State** — `InvestmentCommitteeState.quant_signals` plus a config toggle `enable_quant_signals`.
- **Evidence node** — `qlib_signals_node` runs in the *prepare* phase (once, shared across providers): the ticker’s alpha expected return, universe percentile, and model rank-IC, plus a peer-group factor-risk read (forward vol, factor exposures, minimum-variance weight). It auto-fans-out from the engine and auto-fans-in to every analyst, and degrades to `available: false` on any error, so it never fails a run.

- **Specialist** — the Quantitative Factor Analyst reconciles the statistical alpha/risk signals with the fundamental/DCF thesis and flags alpha-vs-valuation disagreements.

9 REST API and frontend

Router `api/routers/quant.py` (under `/api/quant`), all handlers offloading qlib work to worker threads:

Endpoint	Purpose
GET <code>/backends</code>	Optimizer backends, risk models, alpha-model metadata.
POST <code>/alpha</code>	Expected returns for a universe (or the latest top- N cross-section).
POST <code>/risk</code>	Forward covariance / volatility / factor exposures.
POST <code>/optimize</code>	Portfolio optimization; optimizer selects native or any qlib backend.
POST <code>/backtest</code>	Walk-forward out-of-sample signal backtest with qlib risk_analysis .

The *Quant* tab (`QuantView.tsx`) exposes an optimizer-backend selector, risk-model and μ -source selectors, a live expected-returns table, an optimized-weights table, and a backtest panel.

10 Operations

- **qlib on Python 3.13.** No PyPI wheel exists for 3.13, so qlib is built from a bundled clone and installed non-editable, so `import qlib` resolves from any working directory (the C++ toolchain is required for the build). Lean runtime deps are pinned in `backend/requirements.txt` (`lightgbm`, `scikit-learn`, `cvxpy`, `ecos`, `dill`, `pyarrow`, ...).
- **Cold start / pre-warm.** The first alpha call for a jurisdiction builds the cross-section panel from the warehouse and then caches it (TTL via `QLIB_ALPHA_CACHE_TTL`); the walk-forward backtest is cached separately. Both are pre-warmed on startup so the first user request is served from cache.
- **Retraining.** The alpha model is (re)trained via the CLI: `python -m api.quant.qlib_alpha train -jurisdiction US`; artifacts are persisted per jurisdiction and hot-reloaded on file mtime.
- **Tests.** `backend/api/quant/tests/test_quant.py` covers the optimizer backends, the structured covariance, the exact scoring fallback, the alpha blend, the IC re-split, and a DB-gated panel/train smoke test.

A lean, in-process qlib embed: one point-in-time panel, a learned alpha and a structured covariance, dual optimization, and a walk-forward backtest — wired into the scanner and the committee, and degrading cleanly to the prior behavior wherever a model or dependency is absent.